INFORME DE REMEDIACIÓN · TESINA · PARTE 2

De 9 a 0, sin tocar código

Auditoría de dependencias npm sobre los dos frontends Next.js del sistema — `votacion-frontend-admin` (puerto 8081) y `votacion-frontend-votante` (puerto 8082) — mismo stack, mismos hallazgos.

Fecha: **2026-09-07** Alcance: **frontend-admin + frontend-votante**

Herramienta: **npm audit** Stack: **Next.js 16 · Turbopack**

9 → 0	4	0	0
Vulnerabilidades (los dos proyectos)	Paquetes actualizados	Archivos de código ajustados	Regresiones detectadas

01 · PUNTO DE PARTIDA

Qué encontró `npm audit`

Tres cadenas de dependencias — dos compartidas por ambos proyectos, una exclusiva del admin.

Antes

Después

`next 9.3.4–16.3.0-preview` + `postcss ≤8.5.22` + `sharp <0.35.0` ALTA

`admin` `votante`

- **Next.js (9 CVEs):** bypass de middleware/proxy con Turbopack, DoS en Server Actions, SSRF en Server Actions y en rewrites, confusión de caché de response bodies (2 variantes), payload no acotado en Edge runtime, DoS en Image Optimization vía SVG, exposición de endpoints internos de Server Functions
- **PostCSS:** XSS por `</style>` sin escapar, lectura arbitraria de archivos vía `sourceMappingURL` (2 variantes del mismo fix incompleto), path traversal en auto-carga de source maps
- **sharp:** vulnerabilidades heredadas de `libvips` (4 CVEs 2026)

`qs 2.2.5–6.15.3` MODERADA

admin votante

- Bypass de límite de array vía comas en claves con corchetes
- DoS vía `isBuffer` controlado por el atacante

`browserslist ≤4.28.6` ALTA

admin votante

- Crecimiento de memoria sin límite (sin eviction de caché) vía queries distintas → OOM eventual
- Crash / escritura de prototype vía `browserslist-stats.json` no confiable

02 · EJECUCIÓN

Proceso de remediación

Mismos cuatro pasos en los dos proyectos — se marca el único punto donde divergieron.

1 `npm audit fix` (sin `--force`)

Resuelve `qs` en ambos y `browserslist` en el admin, sin tocar `next`. Quedan 3 vulnerabilidades altas en cada proyecto, todas resueltas por el mismo paquete.

2 Bump manual de Next — sin `--force` real

`next` era la única dependencia directa, fijada exacta en `16.2.10` (`postcss` y `sharp` viven anidados adentro). `npm audit` lo marca como que exige `--force` solo porque el pin exacto queda "fuera de rango" — en los hechos es un bump de **patch dentro de la misma major**, mucho más chico que el de Fastify en el backend.

PAQUETE	ANTES	DESPUÉS	PROYECTOS
`next`	`16.2.10`	`16.3.4`	admin + votante
`eslint-config-next`	`16.2.10`	`16.3.4`	admin + votante (en sync)

3 Verificación: typecheck + build

`npm install` → 0 vulnerabilidades en los dos. El admin tipó y compiló limpio a

la primera. El votante tiró un error en `.next/dev/types/validator.ts` — no era código propio, sino caché vieja de rutas tipadas de una corrida anterior con Next 16.2. Se resolvió borrando `.next` (con alguna traba de rutas largas típica de una carpeta sincronizada con OneDrive, pero sin afectar el resultado).

4 Build de producción + smoke test en dev

Los dos compilan sin error y levantan bien. Rutas clave probadas con el backend real corriendo:

```
admin (8081)> /  /login  /admin
200  200  200

votante (8082)> /  /login  /votante  /soporte
200  200  200  200
```

A diferencia del backend — donde el bump de Fastify obligó a tocar 3 archivos (tipos de `setErrorHandler`, `loggerInstance`, retorno de `buildApp`) — acá no hizo falta cambiar una sola línea de código propio. Fue enteramente un bump de versión.

03 · CIERRE

Estado final

votacion-frontend-admin

`localhost:8081`

5 → 0 VULNERABILIDADES

✓ typecheck limpio
✓ build: 8 rutas compiladas
✓ /, /login, /admin → 200

votacion-frontend-votante

`localhost:8082`

4 → 0 VULNERABILIDADES

✓ typecheck limpio (tras limpiar .next)
✓ build: 6 rutas compiladas
✓ /, /login, /votante, /soporte → 200

Con esto, los tres proyectos del sistema (backend, frontend admin, frontend votante) quedan en 0 vulnerabilidades reportadas por `npm audit`. Ver el informe del backend para la parte con cambios de código real.

frontends @ 2026-09-07